

anymcp — PITCH & DEMO SCRIPT (read-along)

Everything runs **locally on your laptop** — no wifi/cloud dependency. Every command below is copy-paste. Format: **[DO]** = what you do on screen · **[SAY]** = read this out loud. Read the SAY lines almost verbatim; they're written to sound natural.

THE ONE-LINE POSITIONING (memorize this)

"anymcp turns any API into an agent-ready MCP server — from just its docs, no spec required — and hands your agent one connection that unlocks all of them."

The two things that make people lean in:

1. **No spec needed** — competitors need a `swagger.json`; we read the HTML docs like a developer would.
 2. **The hub** — connect an agent to ONE endpoint and it can use *every* API you've wired in.
-

PRE-FLIGHT (do this BEFORE you share your screen)

```
cd C:\Users\karti\anymcp
# 1. load secrets + force local store, in THIS terminal:
Get-Content .env | ? { $_ -match '^\s*[^#].*=' } | % { $kv=$_ -split '=',2; if ($kv[0].T
# 2. short command alias:
function anymcp { npx tsx src\use-cli.ts @args }
# 3. start the web UI + gateway (their own windows):
Start-Process powershell -ArgumentList '-NoExit','-Command',"cd '$PWD'; npx tsx web\serv
Start-Process powershell -ArgumentList '-NoExit','-Command',"cd '$PWD'; npx tsx src\gate
```

- Open <http://localhost:5173> in a browser tab (leave it ready).
- Pre-warm once so first click is instant: `anymcp tools coingecko`
- Pre-run the Inspector once so it's cached: `npx @modelcontextprotocol/inspector --cli node --import tsx src\hub.ts --method tools/list`
- (Optional) In Cursor, pre-add `anymcp-hub` from `demo/cursor-mcp.json` and confirm it lists tools.
- Close noisy apps/notifications. **Share the specific window, not your whole desktop** (so your notes stay private).

Checklist before you talk: browser tab on 5173 ✓ · one terminal with `anymcp` alias + env ✓ · gateway window up ✓ · Inspector cached ✓.

THE DEMO — 6 beats (~7-9 min)

BEAT 1 — The problem (10 sec, no screen needed yet)

[SAY] "Every company wants agents that can actually DO things — call their APIs. But 90% of APIs have no MCP server, and most don't even have an OpenAPI spec — just documentation. Today, wiring each one up is a custom engineering project. *anymcp* removes that entirely."

BEAT 2 — Generate an MCP from nothing but docs (the visual "aha") → browser

[DO] Switch to the browser tab (<http://localhost:5173>). In the left panel, the docs URL is pre-filled → click **Generate** →. **[SAY]** "I'm giving it a plain documentation page — no spec. Watch: it crawls the page, cleans it, reads it with a model, and generates a working MCP server with typed tools, auth, and error handling — live." **[DO]** Let the stages stream, then the generated-server card appears with typed tools. **[SAY]** "That's a real MCP server. There was never an OpenAPI file — it read the docs the way one of your engineers would."

[DO] (Optional, 10 sec) Click **Spec: httpbin** → **Generate** → — stages become *Load spec* → *Convert* → *Generate*, badge shows "from OpenAPI spec · no LLM", 73 tools. **[SAY]** "And if you DO have a spec — OpenAPI or Swagger — we use it directly, no guessing. So we're a superset: spec-optional, not spec-required. Docs, OpenAPI, Swagger — whatever you have, we take it."

BEAT 2c (enterprise, optional but strong) — turn a real business system into agent tools

[DO] In the web UI, paste an enterprise spec and Generate → (pre-tested, reliable):
`https://api.apis.guru/v2/specs/googleapis.com/calendar/v3/openapi.json` → **Calendar API, 37 tools**. (Or Slack: `.../slack.com/1.7.0/openapi.json` → 174 tools.) **[SAY]** "This is Google Calendar — 37 agent-ready tools in seconds. Same for Slack, Jira, your CRM, or your own internal APIs. Every system your business runs on becomes something your agents can operate — no integration project, no waiting for a vendor to ship an MCP server."

BEAT 3 — Prove it's a REAL, standard MCP server → terminal

[DO] Switch to the terminal. Run:

```
npx @modelcontextprotocol/inspector --cli node --import tsx .\out\chuck-demo\server.ts
```

[SAY] "This isn't my tool inspecting my tool — this is the official MCP Inspector, the reference client. If it can drive our generated server, so can any agent: Claude, Cursor, anything that speaks MCP."

BEAT 4 — THE HUB: one connection, every API (the differentiator) → terminal / Cursor

[DO] Run:

```
npx @modelcontextprotocol/inspector --cli node --import tsx .\src\hub.ts --method tools,
```

Scroll the list — `coingecko__...`, `frankfurter__...`, `httpbin__...`, `chuck__...`. [SAY] "This is the *anymcp* hub — a single MCP endpoint. Notice: with ONE connection, an agent now sees the tools of every API we've wired in — dozens of them, namespaced by source. Your agent connects once and can suddenly do everything." [DO] Call one through the hub:

```
npx @modelcontextprotocol/inspector --cli node --import tsx .\src\hub.ts --method tools,
```

[SAY] "Live Bitcoin price — routed through the hub to the right API, in real time. For a user this means: point your agent at *anymcp* once, and every API you've ever added just works."

[DO] (Strong alternative to the two Inspector calls — if Cursor is set up) Switch to **Cursor** with `anymcp-hub` enabled, and type in chat: "What's the price of Bitcoin, and tell me a Chuck Norris joke about science." [SAY] "A real AI IDE — not something I built — connected to the hub, using two different APIs in one request. Zero custom integration. That's the whole promise: any agent, any API, one connection."

[DO] (Terminal-only alternative to Cursor — an agent in your shell, verified working) Run:

```
anymcp ask "What's the price of bitcoin, and give me a Chuck Norris joke about science?"
```

It prints which tools it calls (`coingecko__...`, `chuck__...`) then answers. [SAY] "Same thing from the command line — an agent, one hub connection, calling two different APIs to answer one question." (If it stalls, Ctrl-C and fall back to the two Inspector calls above.)

BEAT 5 — Add ANY API the audience names, live → terminal

[SAY] "Let's make a brand-new one right now. Give me any public API with online docs." [DO] Run (swap in their URL; multiple URLs if the base URL is on a separate page):

```
anymcp new https://THEIR-API.com/docs
```

[SAY] "One command: it's reading the docs, extracting the endpoints, generating the server, and listing the tools — and it's immediately usable." [DO] Call a tool on it: `anymcp call --dir .\out\<name> <tool> key=value`

If it stalls or the base URL comes back empty: don't fight it — "here's one I made earlier" and use `.\out\chuck-demo` (or `coingecko`). Keep moving; never debug live.

BEAT 6 — How a customer actually uses it (30 sec, talk over the terminal)

[SAY] "Two ways to run this. **Self-serve/local**: a developer installs `anymcp`, points it at their docs, and their agent connects locally — nothing leaves their machine. **Hosted**: we run the hub in the cloud, they get an API key, and their agent connects to a URL — every credential encrypted at rest, every API sandboxed in its own process. Same product, their choice of deployment."

THE CLOSE (memorize)

[SAY] "So: any API — from docs, OpenAPI, or Swagger — becomes an agent-ready MCP server in seconds, and one hub connection gives an agent all of them. We turn the long tail of APIs that will never get an MCP server into something your agents can use today. [Then your ask: pilot / design partner / investment.]"

Q&A — objections & the honest answers

"Do I need to show the frontend?" Not required — the *hub* + *agent* flow is the real pitch. But the frontend is a great 30-second "aha" because the streaming generation is visually convincing. **Investors**: show frontend + story first, then terminal for credibility. **Technical buyers**: lead with the terminal/hub (that's the substance), frontend is a nicety.

"How does it load on a user's PC / how do I host it?"

- **Local**: `npx anymcp ...` (needs Node.js). Their agent connects over a stdio config — a few lines in Claude Desktop / Cursor / their runtime. Nothing hosted on our side; nothing leaves their machine.
- **Hosted (SaaS)**: we run the gateway/hub; the user gets an API key and points their agent at a URL (`Authorization: Bearer <key>`). Credentials are encrypted at rest (AES-256-GCM), each API runs sandboxed in its own subprocess, isolated per customer.
- **For this demo**: it's all running on my laptop in "hosted style." Don't claim a live public cloud URL you can't open — say "*this is the same code we deploy; today it's local so it works without wifi.*"

"Is it really not using a spec?" Correct — Beat 2 generates from an HTML docs page. If a spec exists we use it (deterministic, no LLM); if not, we read the docs. That long tail — APIs with docs but no spec — is the wedge.

"What about auth / private APIs?" The generated server injects the API's credential; hosted, the gateway fetches and injects it per-request, encrypted at rest, never exposed to the agent. (Demo APIs are public, so there's nothing to type.)

"What's the moat?" The spec-less extraction quality + verification, and the hub as the aggregation layer agents standardize on. The more APIs wired in, the more valuable one connection becomes.

TELEPROMPTER — reading this while screen-sharing (you have two screens)

Simplest, most reliable (recommended):

1. Share only the **app/browser/terminal window**, NOT your entire desktop (Zoom/Meet/Teams all let you pick a window). Then your notes on the other screen are never visible.
2. Put THIS file, large-font, on your **second (unshared) screen**:
 - Open it in VS Code → `Ctrl+K V` (markdown preview) → `Ctrl +` to zoom up. Scroll as you go. Or
 - Any markdown reader / even the browser.

If you want a scrolling teleprompter feel: use a free browser teleprompter — cueprompter.com (no install, paste the SAY lines) or a similar web teleprompter. **Test it once beforehand** on your setup; don't trust it live.

Bonus: if you present the deck (`demo/pitch-deck.html`), keep this script on screen 2 as speaker notes.

⚠ The safest habit regardless of tool: **share a window, not the whole screen**. That alone guarantees your notes never leak, whatever you read them from.